

Computer Science (Degree)

CTSE 2025 - Concepts and Revision Notes

- Companion volume to the three question-and-answer documents.
- Covers the underlying concepts of each subject area, not only the topics the 2025 papers happened to ask about.
- Organised as thirteen chapters, each ending with a list of the facts most often tested.

How to use this document

- Read a chapter before attempting the corresponding questions in the paper documents, then use the 'Most often tested' list at the end of each chapter as a final revision pass.
- Definitions are given in the form an examiner expects: a one-sentence definition first, then the elaboration.
- Formulas and numeric facts worth memorising are set in monospace so they are easy to find when skimming.
- Where two concepts are habitually confused, they appear side by side under a 'Do not confuse' heading. Those pairs are where marks are most commonly lost.

1. Computer Fundamentals and Architecture

The von Neumann model

A stored-program computer holds both instructions and data in the same read-write memory. This is what makes a computer general purpose: changing the program changes the machine's behaviour without rewiring it.

- Five functional units: input, memory, ALU, control unit, output. ALU + control unit + registers = CPU.
- Three buses: address (which location), data (the value), control (what operation, and timing).
- The von Neumann bottleneck is the limited bandwidth of the single CPU-memory path. Mitigations: cache hierarchy, wider buses, prefetching, and the Harvard architecture, which uses separate instruction and data memories.

The instruction cycle

- Fetch: the address in the PC is placed in the MAR, memory is read into the MDR, the instruction moves to the IR, and the PC is incremented.
- Decode: the control unit interprets the opcode and works out the addressing mode.
- Execute: operands are fetched, the ALU operates, and the result is stored.
- Interrupt check at the end of each cycle: if an interrupt is pending, the state is saved and control transfers to the handler.

Registers worth naming

- PC, program counter - address of the next instruction.
- IR, instruction register - the instruction currently executing.
- MAR, memory address register - the address currently on the bus.
- MDR or MBR, memory data or buffer register - the data in transit.
- Accumulator - ALU operand and result.
- SP, stack pointer, and general-purpose registers R0..Rn.
- PSW or flags register - zero, carry, sign, overflow, interrupt-enable, mode bits.

Performance

$$\begin{aligned}\text{CPU time} &= \text{instruction count} \times \text{CPI} \times \text{clock cycle time} \\ &= (\text{instruction count} \times \text{CPI}) / \text{clock rate}\end{aligned}$$

$$\text{MIPS} = \text{clock rate} / (\text{CPI} \times 10^6)$$

$$\text{Speedup} = \text{old time} / \text{new time}$$

$$\begin{aligned}\text{Amdahl's law} &= 1 / ((1 - p) + p/s) \\ p &= \text{fraction improved, } s = \text{speedup of that fraction}\end{aligned}$$

Amdahl's law is the reason parallelism has limits: if ten per cent of a program is inherently serial, no number of processors can give more than a tenfold speedup.

Pipelining

- Classic five stages: IF, ID, EX, MEM, WB. Throughput approaches one instruction per cycle; latency per instruction does not fall.
- Hazards: structural, a resource conflict; data, an operand not yet written, cured by forwarding or stalling; control, a branch whose outcome is unknown, cured by prediction, delayed branches or flushing.
- Superscalar issues several instructions per cycle. Out-of-order execution reorders around stalls. Both are complementary to pipelining rather than alternatives.

Cache

- Mapping: direct - each block has exactly one line, cheap but conflict prone. Fully associative - any line, best hit rate, expensive comparators. Set-associative - k lines per set, the practical compromise.
- Replacement: LRU, FIFO, random. Write policy: write-through, simple and consistent, versus write-back, less traffic but needs a dirty bit.

Average access time = hit time + miss rate x miss penalty

Effective access time with TLB

$$= h(TLB + mem) + (1-h)(TLB + 2 \times mem)$$

- The three Cs of misses: compulsory, the first ever reference; capacity, the working set exceeds the cache; conflict, too many blocks map to one set.

Addressing modes

- Immediate, register, direct, indirect, register indirect, indexed, base plus displacement, relative, implied, autoincrement and autodecrement.
- Match the mode to the use: immediate for constants, register indirect for pointers, indexed for arrays, base plus displacement for record fields and stack frames, relative for branches and position-independent code.

Most often tested

- PC holds the next instruction address; IR holds the current instruction.
- Cache hierarchy is the answer to the von Neumann bottleneck.
- Encoder: 2^n inputs to n outputs. Decoder: n to 2^n . Multiplexer: many to one. Demultiplexer: one to many.
- Ripple carry adder delay = number of stages x per-stage delay.
- Microprogrammed control stores control words in control memory; hardwired control uses fixed logic and is faster but inflexible.
- Memory-mapped I/O gives devices ordinary memory addresses; isolated I/O uses separate IN and OUT instructions.

2. Number Systems and Digital Logic

Number systems and conversion

- Bases: binary 2, octal 8, decimal 10, hexadecimal 16. One hex digit is four bits; one octal digit is three bits, which is why conversion between binary and those bases is done by grouping.
- Decimal to binary: repeated division by 2, reading remainders bottom-up. Fraction: repeated multiplication by 2, reading integer parts top-down.

Signed representations of an 8-bit value

- Sign and magnitude - one sign bit; two representations of zero; arithmetic awkward.
- One's complement - invert all bits; still two zeros; needs end-around carry.
- Two's complement - invert and add one. Single zero, and subtraction becomes addition, so hardware is simpler. This is what real machines use. Range for n bits: $-2^{(n-1)}$ to $2^{(n-1)} - 1$.

Two's complement of 1101 0010:

invert -> 0010 1101

add 1 -> 0010 1110

- Excess-k, or biased, representation is used for floating-point exponents.
- BCD encodes each decimal digit in four bits; Gray code changes exactly one bit between successive values, which is why it is used on shaft encoders and in Karnaugh maps.

Binary to Gray: $G(\text{msb}) = B(\text{msb})$, $G_i = B_{i+1} \text{ xor } B_i$

Gray to binary: $B(\text{msb}) = G(\text{msb})$, $B_i = B_{i+1} \text{ xor } G_i$

IEEE 754 floating point

- Single precision, 32 bits: 1 sign, 8 exponent biased by 127, 23 mantissa. Double precision, 64 bits: 1, 11 biased by 1023, 52.
- Value = $(-1)^s \times 1.\text{mantissa} \times 2^{(\text{exponent} - \text{bias})}$ for normalised numbers.

Boolean algebra and minimisation

- Operators: AND, OR, NOT; derived NAND, NOR, XOR, XNOR. NAND and NOR are each universal.
- Laws: identity, null, idempotent, complement, involution, commutative, associative, distributive, absorption, consensus, and De Morgan.

De Morgan: $(A + B)' = A' . B'$ $(A . B)' = A' + B'$

- Canonical forms: SOP, a sum of minterms, and POS, a product of maxterms. Any function can be written in either.
- Karnaugh maps minimise up to about six variables. Cells are arranged in Gray-code order so that adjacent cells differ in one variable; group powers of two, use the largest groups, allow wrap-around, and treat don't-cares as whichever value helps.
- The Quine-McCluskey method is the tabular, algorithmic alternative, suitable for more variables and for automation.

Combinational building blocks

- Half adder: $S = A \text{ xor } B$, $C = A . B$. Full adder: $S = A \text{ xor } B \text{ xor } C_{in}$, $C_{out} = A . B + C_{in} . (A \text{ xor } B)$.

- Ripple carry adder - simple, delay proportional to width. Carry look-ahead - computes carries in parallel from generate $G_i = A_i B_i$ and propagate $P_i = A_i \oplus B_i$.
- Subtractor - two's complement the subtrahend and set the first carry-in to 1.
- Multiplexer 2^n to 1 with n select lines; demultiplexer 1 to 2^n ; decoder n to 2^n ; encoder 2^n to n ; priority encoder resolves simultaneous inputs.
- Comparator, parity generator and checker, code converters.
- Signed overflow is detected as carry-in to the sign bit XOR carry-out of it.

Sequential building blocks

- Latch - level sensitive. Flip-flop - edge triggered.
 - SR: $S=R=1$ forbidden
 - D: $Q(\text{next}) = D$
 - JK: $Q(\text{next}) = J \cdot Q' + K' \cdot Q$
 - T: $Q(\text{next}) = T \oplus Q$
- Race-around: a level-triggered JK with $J=K=1$ toggles repeatedly while the clock is high. Cured by master-slave or by edge triggering.
- Counters: asynchronous or ripple, simple but with cumulative delay; synchronous, all flip-flops clocked together; up, down, up/down; modulo- N ; ring counter with n states; Johnson or twisted-ring with $2n$ states.
- Shift registers: SISO, SIPO, PISO, PIPO; also universal and bidirectional.
- Design procedure for a synchronous sequential circuit: state diagram, state table, state assignment, excitation table using the flip-flop's excitation equations, K-map minimisation, then the logic diagram.
- Moore machine - output depends on state only. Mealy machine - output depends on state and input, so it reacts one cycle sooner and usually needs fewer states.

Most often tested

- Two's complement = invert then add one. Forgetting the increment is the classic error.
- $A + (AB)' = 1$, by De Morgan then complement law.
- Ring counter with n flip-flops has n states, not 2^n .
- Sequential output depends on current inputs and past states; combinational on current inputs only.
- Race-around belongs to the JK; the edge-triggered D avoids it by construction.

3. Programming in C and C++

Types, storage and qualifiers

- Typical sizes: char 1, short 2, int 4, long 8, float 4, double 8 bytes. Only char is fixed at 1 by the standard; the rest are implementation defined with minimum ranges.
- Storage classes: auto, the default for locals; register, a hint; static, lifetime of the program with internal linkage at file scope; extern, defined elsewhere.
- A static local variable has function scope but program lifetime - the distinction between scope and lifetime is examined constantly.
- const - the value may not be modified through that name. volatile - the value may change outside the program's control, so the compiler must not cache or elide accesses. The two are independent and can be combined.

Integer division and promotion

- If both operands are integers the result is an integer, truncated towards zero: 5/2 is 2. One operand must be floating point to get 2.5.
- Usual arithmetic conversions promote the narrower operand. Mixing signed and unsigned is a common source of bugs, because the signed value converts to unsigned.

Operators and precedence

- From highest: postfix, then unary, then * / %, then + -, then shifts, then relational, then equality, then bitwise AND, XOR, OR, then logical AND, OR, then conditional, then assignment, then comma.

```
10 + 5 * 2 / 4 - 1
  = 10 + (10 / 4) - 1      multiplication and division first, left to right
  = 10 + 2 - 1            integer division truncates
  = 11
```

- Short-circuit evaluation: && stops at the first false, || at the first true. This is what makes if (p != NULL && p->x) safe.

Pointers

- Declaration int *p; initialisation p = &x; dereference *p; arithmetic scaled by sizeof(type).
- Array-pointer equivalence: arr[i] is *(arr + i). An array name decays to a pointer to its first element, which is why array size is not available inside a function.
- Function pointers: int (*fp)(int, int); used for callbacks and dispatch tables.
- Common failures: dereferencing NULL or an uninitialised pointer, which is undefined behaviour; a dangling pointer after free; a memory leak from a lost pointer; double free; and off-by-one buffer overrun.
- Partial initialisation zero-fills: int a[5] = {1,2}; gives {1,2,0,0,0}.

Dynamic memory

- C: malloc, which does not initialise; calloc, which zeroes; realloc, which resizes; free. Always check the returned pointer for NULL.
- C++: new and delete, new[] and delete[]. Prefer smart pointers - unique_ptr and shared_ptr - so that ownership is explicit and release is automatic.

Functions and parameter passing

- Prototype declares return type, name and parameter types so calls can be checked before the definition is seen.
- Call by value copies the argument. Call by reference passes an address, in C explicitly with a pointer, in C++ also with a reference parameter.
- Recursion needs a base case and progress towards it. Each call consumes a stack frame, so depth is bounded; tail recursion may be optimised into a loop.

C++ additions worth knowing

- The four pillars: encapsulation, abstraction, inheritance, polymorphism.
- Compile-time polymorphism through overloading and templates; run-time polymorphism through virtual functions and a vtable.
- Constructors, destructors, copy constructor, copy assignment, move constructor, move assignment - the rule of five.
- enum class gives scoped, strongly typed enumerators that do not implicitly convert to int.
- constexpr evaluates at compile time; inline suggests inlining; explicit prevents unwanted implicit conversion.
- STL containers - vector, list, deque, set, map, unordered_set, unordered_map - with iterators and algorithms such as sort, find and reverse.
- RAII: acquire a resource in a constructor and release it in a destructor, so cleanup happens even when an exception unwinds the stack.

Most often tested

- char is the smallest type, exactly one byte by definition.
- malloc is declared in <stdlib.h>.
- strcmp returns 0 when strings are equal - not 1.
- volatile stops compiler optimisation of accesses; it does not provide atomicity or thread safety.
- $5/2$ is 2; $10 + 5*2/4 - 1$ is 11.
- Dereferencing an uninitialised or NULL pointer is undefined behaviour.

4. Data Structures

Classification

- Linear: array, linked list, stack, queue. Non-linear: tree, graph. Hash-based: hash table.
- Static, with size fixed at compile time, versus dynamic, growing at run time.

Array versus linked list

	Array	Linked list
Access by index	$O(1)$	$O(n)$
Search (unsorted)	$O(n)$	$O(n)$
Insert at head	$O(n)$	$O(1)$
Insert at position	$O(n)$	$O(1)$ given the pointer
Delete	$O(n)$	$O(1)$ given the pointer
Memory	contiguous	scattered + pointer overhead
Cache behaviour	excellent	poor
Size	fixed	dynamic

Stack and queue

- Stack, LIFO, one end: push, pop, peek. Applications - call stack and recursion, expression conversion and evaluation, undo, backtracking, DFS, parenthesis matching.
- Queue, FIFO, two ends: enqueue at rear, dequeue at front. Applications - scheduling, spooling, BFS, buffering.
- Variants: circular queue, which reuses vacated array slots; deque, both ends; priority queue, removal by priority, normally implemented as a binary heap with $O(\log n)$ insert and extract.
- Infix to postfix conversion and postfix evaluation are both stack algorithms and are frequently asked.

Trees

- Terms: root, parent, child, sibling, leaf or external, internal, degree, level, height, depth, subtree, forest.
- Binary tree - at most two children. Full - every node has 0 or 2 children. Complete - all levels filled except possibly the last, filled left to right. Perfect - all leaves at the same depth. Balanced - subtree heights differ by at most one.

$$\begin{aligned}\text{Max nodes at level } l &= 2^l \\ \text{Max nodes in a tree of height } h &= 2^{(h+1)} - 1 \\ \text{Min height for } n \text{ nodes} &= \text{floor}(\log_2 n)\end{aligned}$$

- Traversals: preorder root-left-right, used for copying and for prefix expressions; inorder left-root-right, which yields sorted order in a BST; postorder left-right-root, used for deletion and for postfix evaluation; level order, which uses a queue.
- BST - left subtree less than the node, right subtree greater. Search, insert and delete are $O(h)$: $O(\log n)$ when balanced, $O(n)$ when degenerate, which is what happens if sorted data is inserted in order.
- Self-balancing trees: AVL, where subtree heights differ by at most one, giving stricter balance and faster lookup; red-black, with looser balance and cheaper insertion, used in most standard libraries; B-tree and B+-tree, which are multiway and shallow, designed for disk and used for database indexes.
- Heap - a complete binary tree with the heap property, parent no smaller, or no larger, than its children. Insert and delete are $O(\log n)$, building from an array is $O(n)$, and it backs the priority queue and heap sort.

- Trie - a prefix tree over an alphabet, giving $O(m)$ lookup for a key of length m , used for dictionaries and autocomplete.

Graphs

- Directed or undirected, weighted or unweighted, cyclic or acyclic, connected or disconnected, dense or sparse.
- Representations: adjacency matrix, $O(V^2)$ space, $O(1)$ edge test, good for dense graphs; adjacency list, $O(V + E)$ space, good for sparse graphs, which is the usual case.
- BFS uses a queue, gives shortest paths in an unweighted graph, and is $O(V + E)$ with an adjacency list. DFS uses a stack or recursion, and is used for cycle detection, topological sort and finding connected components.
- Shortest path: Dijkstra for non-negative weights, greedy, $O((V+E) \log V)$ with a heap; Bellman-Ford, which tolerates negative weights and detects negative cycles, $O(VE)$; Floyd-Warshall for all pairs, $O(V^3)$.
- Minimum spanning tree: Prim, which grows one tree, and Kruskal, which sorts edges and uses union-find. Both greedy.
- Topological sort of a DAG, by Kahn's algorithm or by DFS finish times.

Hashing

- A hash function maps a key to a bucket index, ideally uniformly. Load factor = entries / buckets.
- Collision resolution: separate chaining, a list or tree per bucket; open addressing with linear probing, which suffers primary clustering, quadratic probing, or double hashing.
- Average $O(1)$ insert, search and delete; worst case $O(n)$ when everything collides. Rehashing into a larger table when the load factor exceeds a threshold keeps the average bound.
- Used for symbol tables, caches, sets and dictionaries. Note that a hash table has no ordering, so it cannot support range queries - that is when a balanced tree is preferred.

Most often tested

- Stack push and pop are $O(1)$ in both array and linked-list implementations.
- Insertion at the head of a singly linked list is $O(1)$.
- A node with no children is a leaf.
- Postorder is left, right, root.
- Hash table is the standard symbol-table implementation, for $O(1)$ average lookup.
- Separate chaining and open addressing are the two collision-resolution families.
- BFS and DFS are both $O(V + E)$ with an adjacency list, $O(V^2)$ with a matrix.

5. Algorithms and Complexity

Asymptotic notation

$O(g)$ upper bound $f(n) \leq c \cdot g(n)$ for $n \geq n_0$
 $\Omega(g)$ lower bound $f(n) \geq c \cdot g(n)$ for $n \geq n_0$
 $\Theta(g)$ tight bound $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$
 $o(g)$ strict upper bound
 $\omega(g)$ strict lower bound

- Hierarchy: $O(1) < O(\log n) < O(\sqrt{n}) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$.
- Best, average and worst case are different questions from O , Ω and Θ - a common confusion. You can state a Θ bound on the worst case.
- Space complexity counts total memory as a function of input size; auxiliary space excludes the input.
- Amortised analysis averages cost over a sequence of operations - dynamic array append is $O(1)$ amortised though an individual resize is $O(n)$.

Recurrences

Master theorem for $T(n) = a \cdot T(n/b) + f(n)$, let $c = \log_b(a)$

$f(n) = O(n^{c-\epsilon}) \rightarrow T(n) = \Theta(n^c)$

$f(n) = \Theta(n^c) \rightarrow T(n) = \Theta(n^c \log n)$

$f(n) = \Omega(n^{c+\epsilon}) \rightarrow T(n) = \Theta(f(n))$ (regularity permitting)

Examples

Merge sort $T(n) = 2T(n/2) + O(n) \rightarrow \Theta(n \log n)$

Binary search $T(n) = T(n/2) + O(1) \rightarrow \Theta(\log n)$

Strassen $T(n) = 7T(n/2) + O(n^2) \rightarrow \Theta(n^{2.81})$

Design paradigms

- Divide and conquer - split, solve recursively, combine. Sub-problems independent. Merge sort, quick sort, binary search, Strassen, closest pair.
- Dynamic programming - overlapping sub-problems plus optimal substructure, solved once and stored, either top-down with memoisation or bottom-up with a table. Fibonacci, 0/1 knapsack, LCS, matrix chain, Floyd-Warshall, edit distance.
- Greedy - locally optimal irrevocable choice at each step. Correct only with the greedy-choice property. Dijkstra, Prim, Kruskal, Huffman, activity selection, fractional knapsack.
- Backtracking - systematic search with pruning when a partial solution cannot be completed. N-queens, sudoku, subset sum, graph colouring.
- Branch and bound - backtracking for optimisation problems, using a bound to prune. Travelling salesman, 0/1 knapsack.
- Randomised - randomised quick sort, Monte Carlo and Las Vegas algorithms.

Do not confuse

- Divide and conquer versus dynamic programming: the sub-problems are independent in the former and overlapping in the latter. That single difference is why one memoises and the other does not.
- Greedy versus dynamic programming: greedy commits to one choice, DP considers all and keeps the best. Fractional knapsack is greedy; 0/1 knapsack needs DP.

Sorting

Algorithm	Best	Average	Worst	Space	Stable
Bubble	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	yes
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	no
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	yes
Merge	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	yes
Quick	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	no
Heap	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	no
Counting	$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(k)$	yes
Radix	$O(d(n+k))$	$O(d(n+k))$	$O(d(n+k))$	$O(n+k)$	yes

- Any comparison sort needs $\Omega(n \log n)$ comparisons in the worst case, by a decision-tree argument. Counting and radix sort beat that only because they do not compare - they exploit the structure of the keys.
- Stability means equal elements keep their relative order. It matters when sorting by a second key after a first.
- Quick sort's worst case arises on sorted input with a first or last pivot; randomised or median-of-three pivots avoid it in practice.

Searching

- Linear search $O(n)$, works on unsorted data. Binary search $O(\log n)$, requires sorted data. Interpolation search $O(\log \log n)$ on uniformly distributed sorted data. Hash lookup $O(1)$ average.

Complexity classes

- P - decidable in polynomial time by a deterministic machine.
- NP - a proposed solution can be verified in polynomial time. P is a subset of NP. NP does not mean 'not polynomial'.
- NP-hard - at least as hard as every problem in NP, not necessarily a decision problem.
- NP-complete - in NP and NP-hard. SAT, 3-SAT, travelling salesman, clique, vertex cover, Hamiltonian cycle, subset sum, graph colouring.
- Cook-Levin theorem: SAT is NP-complete. A polynomial algorithm for any NP-complete problem would give one for all of them, so P would equal NP.
- Practical responses to intractability: approximation algorithms with a proven ratio, heuristics, parameterised algorithms, and restriction to tractable special cases.

Most often tested

- $f(n) = O(g(n))$ means $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$.
- Worst case is quoted because it is a guarantee.
- Merge sort is the divide-and-conquer sort with an explicit combine step; quick sort has none.
- Fibonacci is the canonical dynamic-programming example.
- Greedy chooses what is locally optimal at the current moment.
- NP means verifiable in polynomial time.

6. System Software: Compilers, Assemblers, Linkers, Loaders

The translation pipeline

source -> preprocessor -> compiler -> assembler -> object file
-> linker -> executable -> loader -> running process

- Preprocessor - macro expansion, file inclusion, conditional compilation.
- Compiler - source language to assembly or object code, reporting errors.
- Assembler - assembly to relocatable machine code. A two-pass assembler builds the symbol table in pass one and generates code in pass two.
- Linker - combines modules, resolves external references, relocates addresses, pulls in library members.
- Loader - brings the image into memory, performs final relocation if needed, and transfers control to the entry point.

Do not confuse

- Compiler translates the whole program before execution and reports all errors; interpreter translates and executes statement by statement and stops at the first error. A hybrid such as Java compiles to bytecode, then interprets or JIT-compiler it.
- Linker resolves references between modules at build time; loader places the image in memory at run time. A linking loader does both at load time.

Compiler phases

- Lexical analysis - characters to tokens, using finite automata; Lex or Flex; detects illegal characters and unterminated literals.
- Syntax analysis - tokens to a parse tree, using a context-free grammar; Yacc or Bison; detects structural errors.
- Semantic analysis - type checking, declaration checking, coercion insertion; produces an annotated tree; detects type mismatch and undeclared identifiers.
- Intermediate code generation - three-address code, quadruples, triples, SSA. Decouples front end from back end, so m languages and n targets need m + n parts.
- Code optimisation - constant folding and propagation, common sub-expression elimination, dead-code elimination, copy propagation, strength reduction, loop-invariant code motion, loop unrolling.
- Code generation - instruction selection, register allocation by graph colouring, instruction scheduling.
- Spanning all phases: the symbol table, normally a hash table; and the error handler, with panic-mode, phrase-level, error-production and global-correction recovery.

Parsing

- Top-down: recursive descent, which uses the implicit call stack; predictive LL(1), which uses a table plus an explicit stack and requires a grammar with no left recursion and no common prefixes.
- Bottom-up: shift-reduce, LR(0), SLR(1), LALR(1), canonical LR(1). Uses an explicit stack with ACTION and GOTO tables. More powerful than LL, and what Yacc generates.
- Operator-precedence parsing handles expression grammars using precedence relations.
- Ambiguity, left recursion and left factoring are the standard grammar transformations asked about.

Most often tested

- The linker resolves external references; a linking loader does it at load time.
- LR parsing is the stack-and-table technique - though LL(1) is also table driven.
- Symbol table is usually a hash table.
- Front end is machine independent, back end is machine dependent; the intermediate representation is the boundary.

7. Operating Systems

Functions and types

- Functions: process management, memory management, file management, device management, storage management, protection and security, and the user interface.
- Types: batch, multiprogrammed, time-sharing, real-time - hard and soft, distributed, network, clustered, embedded, mobile.
- Generations: 1 vacuum tubes with no OS; 2 transistors with batch; 3 integrated circuits with multiprogramming and time-sharing; 4 VLSI microprocessors with personal and network systems; 5 AI, natural language and cloud.
- Kernel structures: monolithic, layered, microkernel with services in user space, modular, and hybrid. Also virtual machines and containers.
- Dual-mode operation - user mode and kernel mode - with a system call trapping into the kernel, is the hardware basis of protection.

Processes and threads

- Process states: new, ready, running, waiting or blocked, terminated. Also suspended-ready and suspended-blocked when swapping is involved.
- The PCB holds the process id, state, program counter, registers, scheduling and memory information, accounting data, and open files. It is saved and restored on a context switch.
- Thread - the unit of CPU scheduling, sharing code, data and files with its peers but having its own stack, registers and program counter. Cheaper to create and to switch than a process.
- Multithreading models: many-to-one, one-to-one, many-to-many.
- IPC: shared memory, which is fast but needs synchronisation; message passing, which is safer; pipes, sockets, and signals.

CPU scheduling

Turnaround time = completion - arrival
Waiting time = turnaround - burst
Response time = first CPU allocation - arrival
Throughput = processes completed per unit time

- FCFS - non-preemptive, simple, suffers the convoy effect where short jobs wait behind one long one.
- SJF - optimal average waiting time, but burst length must be predicted and long jobs can starve. SRTF is its preemptive form.
- Priority - preemptive or not; starvation is the risk, and ageing is the cure.
- Round Robin - preemptive with a time quantum; fair, starvation free, good response time; quantum too large degenerates to FCFS, too small wastes cycles on context switches.
- Multilevel queue and multilevel feedback queue - several queues with different priorities and quanta, with processes moving between them based on behaviour. This is what real systems use.

Synchronisation and deadlock

- Critical-section requirements: mutual exclusion, progress, bounded waiting.
- Mechanisms: Peterson's software solution, test-and-set and compare-and-swap in hardware, mutex, semaphore - counting and binary, monitor, and condition variables.

- Classic problems: producer-consumer with a bounded buffer, readers-writers, dining philosophers.
- Deadlock requires all four Coffman conditions simultaneously: mutual exclusion, hold and wait, no preemption, circular wait.
- Handling: prevention, by denying one condition; avoidance, using the banker's algorithm with advance knowledge of maximum needs; detection and recovery, using a wait-for graph; or ignore it and reboot, which is what general-purpose systems do.
- Starvation is indefinite postponement, and is distinct from deadlock, where nothing can proceed at all.

Memory management

- Contiguous: single partition, fixed partitions with internal fragmentation, variable partitions with external fragmentation. Placement by first fit, best fit, worst fit, next fit. Compaction cures external fragmentation at a cost.
- Paging - fixed-size pages and frames, no external fragmentation, page table per process, TLB to avoid the double memory access, internal fragmentation of at most half a page per process on average.
- Segmentation - variable-size logical segments matching the programmer's view, natural protection and sharing, but external fragmentation.
- Segmented paging combines both, and is what real architectures provide.
- Virtual memory with demand paging - a page fault brings in a page on reference. Replacement by FIFO, subject to Belady's anomaly, Optimal as an unachievable benchmark, LRU and its clock or second-chance approximations, LFU, MFU.
- Thrashing - more time paging than executing, caused by too few frames for the working set. Cures: the working-set model and page-fault frequency control.
- Belady's anomaly: with FIFO, more frames can produce more page faults. Stack algorithms such as LRU and Optimal do not suffer it.

File systems and I/O

- File attributes, operations, access methods - sequential, direct, indexed.
- Allocation: contiguous, fast but fragmenting; linked, no external fragmentation but no direct access; indexed, with an index block, as in the Unix inode with direct, single, double and triple indirect blocks.
- Free-space management: bit vector, linked list, grouping, counting.
- Directory structures: single level, two level, tree, acyclic graph, general graph.
- Disk scheduling: FCFS, SSTF, SCAN, C-SCAN, LOOK, C-LOOK. Seek time dominates on magnetic disks and is irrelevant on SSDs.
- I/O techniques: programmed I/O with polling, interrupt-driven I/O, and DMA, which transfers blocks without CPU involvement.
- RAID levels: 0 striping, 1 mirroring, 5 distributed parity, 6 double parity, 10 mirrored stripes.

Most often tested

- PCB is a data structure holding process information, not hardware.
- Time quantum limits how long a process runs per turn.
- Polling is an I/O technique; paging, segmentation and swapping are memory techniques.
- ACLs control access to resources.
- Paging gives internal fragmentation; segmentation gives external.

- Logical addresses are generated by the CPU; physical addresses go on the memory bus; the MMU translates between them.

8. Software Engineering and Project Management

Process models

- Waterfall - sequential, documentation heavy, no return to earlier phases. Suits stable, well-understood requirements; fails when requirements change.
- V-model - waterfall with a test level planned against each development level, so verification is designed in.
- Incremental - deliver in successive builds, each adding function.
- Iterative - refine the whole product over repeated cycles.
- Prototyping - build a throwaway or evolutionary prototype to clarify requirements.
- Spiral - Boehm; repeated cycles of planning, risk analysis, engineering and customer evaluation. Risk driven, and the model that explicitly emphasises early customer feedback.
- RAD - short cycles with heavy reuse and user involvement.
- Agile - Scrum, XP, Kanban. Iterative, adaptive, collaborative, working software over documentation.
- DevOps - continuous integration, delivery and deployment, with development and operations in one loop.

Requirements engineering

- Activities: elicitation, analysis, specification, validation, management.
- Functional requirements say what the system does; non-functional requirements say how well - performance, reliability, availability, security, usability, scalability, maintainability, portability. A response-time limit is non-functional.
- The SRS is the agreement between stakeholders and developers on what the software will do. A good SRS is complete, consistent, unambiguous, verifiable, traceable and modifiable.
- Elicitation techniques: interviews, questionnaires, observation, document analysis, workshops, prototyping, use cases and user stories.

Estimation and planning

- WBS - hierarchical, deliverable-oriented decomposition; the hundred per cent rule; work packages of roughly eight to eighty hours; the basis of estimating, scheduling, budgeting and tracking.
- Estimation methods: expert judgement and Delphi; analogy, which uses historical data from similar past projects; bottom-up decomposition; top-down; algorithmic models.

Basic COCOMO	Effort = a x (KLOC) ^b person-months
organic	a=2.4 b=1.05
semi-detached	a=3.0 b=1.12
embedded	a=3.6 b=1.20

PERT Expected = (O + 4M + P) / 6

- The exponent above 1 encodes diseconomy of scale - doubling size more than doubles effort.
- Function points count external inputs, outputs and inquiries plus internal and external files, weighted by complexity, then adjusted. Language independent and countable from requirements, unlike KLOC.
- Scheduling: activity network, critical path, float or slack, Gantt chart, milestones, resource levelling. Brooks's law - adding people to a late project makes it later.

- Risk management: identification, analysis by probability and impact, planning, and monitoring. Responses are avoid, transfer, mitigate, accept.

Design

- Cohesion should be high - a module should do one thing. Coupling should be low - modules should depend on each other as little as possible. This pair is the most-tested design principle.
- Abstraction, modularity, information hiding, separation of concerns.
- Architectural styles: layered, client-server, MVC, microservices, event-driven, pipe and filter, repository.
- SOLID: single responsibility, open-closed, Liskov substitution, interface segregation, dependency inversion.
- UML diagrams: structural - class, object, component, deployment, package; behavioural - use case, sequence, activity, state, collaboration.
- Coupling from worst to best: content, common, external, control, stamp, data. Cohesion from worst to best: coincidental, logical, temporal, procedural, communicational, sequential, functional.

Testing and quality

- Levels: unit, integration, system, acceptance. Integration testing checks the interfaces between modules.
- Approaches: white box, using knowledge of the code - statement, branch, path and condition coverage, cyclomatic complexity; black box, from the specification - equivalence partitioning, boundary value analysis, decision tables, state transition; grey box.
- Types: functional, regression, smoke, sanity, performance, load, stress, security, usability, compatibility, alpha and beta.
- Verification asks whether we are building the product right, by reviews and inspections; validation asks whether we are building the right product, by testing against the requirement.
- QA is process oriented and preventive; QC is product oriented and detective. Testing is part of QC.
- Cyclomatic complexity $V(G) = E - N + 2P$, which also gives the number of independent paths and a lower bound on the number of test cases needed for branch coverage.
- Maintenance types: corrective, adaptive, perfective, preventive. Maintenance is the largest share of lifetime cost.
- CMMI maturity levels: initial, managed, defined, quantitatively managed, optimising.

Most often tested

- Spiral emphasises iteration and early customer feedback.
- Analogy is the estimation method that uses historical data from similar projects.
- SRS is the agreement on what the software will do.
- A response-time constraint is a non-functional requirement.
- Integration testing tests interfaces between modules.
- QA establishes processes and standards; QC finds defects.
- High cohesion, low coupling.

9. Database Management Systems

Architecture and models

- Three-schema architecture: external or view level, conceptual or logical level, internal or physical level.
- Physical data independence - change the internal schema without touching the conceptual. Logical data independence - change the conceptual schema without touching the external schemas and application programs. Logical is the harder to achieve.
- Data models: conceptual, such as ER and enhanced ER; logical or representational, such as relational, network, hierarchical and object oriented; physical.
- A view is a virtual table defined by a query, giving each user a personalised perspective and serving as a security mechanism.
- DBMS advantages over file systems: reduced redundancy, controlled inconsistency, shared data, enforced integrity and standards, backup and recovery, concurrent access, and data independence.

Relational model

- Relation is a table, tuple a row, attribute a column, degree the number of columns, cardinality the number of rows, domain the set of permitted values.
- Keys: super key; candidate key, a minimal super key; primary key, the chosen candidate; alternate key; composite key, spanning columns; foreign key, referencing another relation; surrogate key, with no business meaning.
- Integrity: domain, entity - primary key unique and not null, referential - every foreign key matches an existing primary key or is null, and user-defined.
- Referential actions on delete or update: CASCADE, SET NULL, SET DEFAULT, RESTRICT, NO ACTION.
- Relational algebra: selection, projection, union, set difference, Cartesian product, rename as the primitives; join in its theta, equi, natural, outer and semi forms, intersection and division as derived operations.

Normalisation

- Functional dependency $X \rightarrow Y$: X determines Y. Full, partial and transitive dependencies drive the normal forms.
- 1NF - atomic values, no repeating groups.
- 2NF - 1NF and no partial dependency of a non-key attribute on part of a composite key.
- 3NF - 2NF and no transitive dependency of a non-key attribute on another non-key attribute.
- BCNF - every determinant is a candidate key; stricter than 3NF.
- 4NF removes multi-valued dependencies; 5NF removes join dependencies.
- Purpose: eliminate redundancy and the insertion, update and deletion anomalies. Cost: more tables and therefore more joins, which is why deliberate denormalisation is sometimes chosen for read performance.

SQL

- DDL - CREATE, ALTER, DROP, TRUNCATE, RENAME. DML - SELECT, INSERT, UPDATE, DELETE. DCL - GRANT, REVOKE. TCL - COMMIT, ROLLBACK, SAVEPOINT.
- INSERT is DML, not DDL - a standard trick question.

- Clause order: SELECT, FROM, WHERE, GROUP BY, HAVING, ORDER BY. Evaluation order is FROM, WHERE, GROUP BY, HAVING, SELECT, ORDER BY - which is why a SELECT alias cannot be used in WHERE.
- WHERE filters rows before grouping; HAVING filters groups after.
- Joins: INNER, LEFT, RIGHT, FULL OUTER, CROSS, SELF.
- Aggregates: COUNT, SUM, AVG, MIN, MAX, and their interaction with NULL - aggregates ignore NULL, except COUNT(*).
- NULL is unknown, not zero and not empty; comparisons with it yield unknown, so IS NULL is required.

Transactions and concurrency

- ACID: atomicity, consistency, isolation, durability.
- States: active, partially committed, committed, failed, aborted.
- Concurrency problems: lost update, dirty read, unrepeatable read, phantom read.
- Control: two-phase locking with growing and shrinking phases, which guarantees serialisability; strict 2PL; timestamp ordering; multiversion concurrency control; optimistic validation.
- Isolation levels: read uncommitted, read committed, repeatable read, serialisable - each preventing progressively more of the problems above at progressively greater cost.
- Recovery: log-based with write-ahead logging, deferred and immediate update, checkpoints, and the ARIES analysis, redo and undo phases.

Storage, indexing and query processing

- Index types: primary or clustered, which fixes physical order; secondary or non-clustered; dense and sparse; multilevel; B-tree and B+-tree, which is the standard because all data sits in the leaves and the leaves are linked for range scans; hash index, good for equality but not for ranges; bitmap index, for low-cardinality columns.
- Query processing steps: parsing and translation, optimisation, code generation, execution.
- Optimisation is both logical, pushing selections and projections down and reordering joins, and physical, choosing access paths and join algorithms - nested loop, block nested loop, index nested loop, sort-merge, hash join.
- Cost-based optimisation relies on catalogue statistics - row counts, block counts, distinct values, histograms - so stale statistics are a common cause of a bad plan.
- Distributed databases: fragmentation - horizontal, vertical, mixed; replication; transparency. Advantages are scalability and availability; costs are complexity and a wider attack surface.
- CAP theorem: under a network partition a distributed system must choose between consistency and availability. NoSQL families - key-value, document, column-family, graph - trade differently, and many offer eventual consistency, described as BASE rather than ACID.

Most often tested

- ER is a conceptual model; relational, network and hierarchical are logical models.
- Logical data independence = change the conceptual schema without changing applications.
- INSERT INTO is not DDL.
- A derived attribute is computed from other attributes.
- A composite key uses two or more attributes together.
- Relationships need both primary and foreign keys.

- Query processing steps do not include 'compilation' as a named step.

10. Computer Networks

Models and layers

OSI	TCP/IP	Data unit	Device
7 Application	Application	data	
6 Presentation	"		
5 Session	"		
4 Transport	Transport	segment	-
3 Network	Internet	packet	router
2 Data Link	Network access	frame	switch
1 Physical	"	bit	hub

- Layer functions: physical - bits and media; data link - framing, MAC addressing, error detection, media access; network - IP addressing, routing, fragmentation; transport - process-to-process delivery, ports, reliability; session - dialogue control and checkpointing; presentation - translation, encryption, compression; application - user services.
- Encapsulation adds a header at each descending layer and strips it on the way up.
- Mnemonic bottom-up: Please Do Not Throw Sausage Pizza Away.

Topologies and transmission

- Bus - one backbone, cheapest, single point of failure. Star - central hub or switch, the modern default. Ring - closed loop with token passing. Mesh - every node to every node, $n(n-1)/2$ links, most reliable, most expensive. Tree - hierarchical stars. Hybrid - combinations.
- Media: twisted pair, UTP and STP, cheap and short-range; coaxial; optical fibre, highest bandwidth, lowest attenuation, immune to interference, hardest to tap; wireless - radio, microwave, infrared, satellite.
- Transmission modes: simplex, half duplex, full duplex. Switching: circuit, packet with datagram or virtual circuit, and message.
- Multiplexing: FDM, TDM - synchronous and statistical, WDM, and CDMA.

Data link and MAC

- Error detection: parity, checksum, CRC. Error correction: Hamming code, and retransmission through ARQ.
- Flow control: stop and wait, go-back-N, selective repeat, sliding window.
- Media access: CSMA/CD in Ethernet, CSMA/CA in Wi-Fi, token passing, ALOHA and slotted ALOHA, and controlled polling.
- MAC address - 48 bits, burned into the NIC, flat and non-routable, used within a LAN.

Network layer and addressing

- IPv4 - 32 bits, written as four dotted octets. Classes A, B, C, D for multicast and E reserved. Private ranges 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16.
- Subnet mask separates network from host; CIDR notation gives the prefix length; subnetting divides a network and supernetting aggregates.

Hosts per subnet = $2^{(32 - \text{prefix})} - 2$
(subtract the network and broadcast addresses)

- IPv6 - 128 bits, eight groups of four hex digits, no broadcast, built-in autoconfiguration.

- Supporting protocols: ARP maps IP to MAC; RARP the reverse; ICMP carries errors and is what ping and traceroute use; DHCP assigns addresses; NAT translates private to public.
- Routing: static versus dynamic; distance vector, RIP with hop count, subject to count-to-infinity; link state, OSPF with Dijkstra; path vector, BGP between autonomous systems.
- A hop is one router traversal; TTL is decremented per hop to kill looping packets.

Transport and application

- TCP - connection oriented, three-way handshake, reliable, ordered, flow control by sliding window, congestion control by slow start and congestion avoidance, 20-byte minimum header, byte stream.
- UDP - connectionless, unreliable, unordered, no flow or congestion control, 8-byte header, message oriented, supports broadcast and multicast.
- Well-known ports: FTP 20 and 21, SSH 22, Telnet 23, SMTP 25, DNS 53, DHCP 67 and 68, HTTP 80, POP3 110, IMAP 143, HTTPS 443.
- DNS resolves names to addresses through a hierarchy of root, TLD and authoritative servers, with recursive and iterative resolution and caching by TTL.
- Security: symmetric encryption such as AES; asymmetric such as RSA; hashing such as SHA-256; digital signatures and certificates; TLS; firewalls; IDS and IPS; VPN. Attacks: sniffing, spoofing, man in the middle, denial of service and distributed denial of service, SQL injection, cross-site scripting, phishing.

Most often tested

- Switch is layer 2 and uses MAC; router is layer 3 and uses IP; hub is layer 1.
- Network layer does logical addressing and routing.
- UDP is the fast connectionless protocol.
- Fibre has the highest bandwidth and lowest attenuation.
- Mesh connects every device to every other.
- A protocol is a set of rules governing communication.
- A hop is the passage of a packet through one router.

11. Web Technology

How the web works

- The client-server request-response cycle: the browser resolves the host name by DNS, opens a TCP connection, performs the TLS handshake for HTTPS, sends an HTTP request, receives a response, parses the HTML, requests sub-resources, and renders.
- URL parts: scheme, host, port, path, query string, fragment.
- HTTP methods: GET to retrieve, POST to submit, PUT to replace, PATCH to modify, DELETE to remove, HEAD for headers only, OPTIONS to discover.
- Status codes: 1xx informational, 2xx success - 200 OK, 201 Created, 3xx redirection - 301 permanent, 302 temporary, 304 not modified, 4xx client error - 400, 401, 403, 404, 5xx server error - 500, 502, 503.
- HTTP is stateless, so state is carried by cookies, sessions, tokens such as JWT, or hidden fields.

GET versus POST

	GET	POST
Data in	URL query string	request body
Visible	yes, and in history	no
Length	limited (~2000 chars)	effectively unlimited
Data types	ASCII only	any, incl. binary
Bookmarkable	yes	no
Cached	yes	no by default
Idempotent	yes	no
Use for	search, filters, reads	login, payment, uploads
PHP array	\$_GET	\$_POST

- POST is not encryption. Only HTTPS protects data in transit; POST merely keeps it out of the URL, the history and the server logs - which is exactly why credentials must never go by GET.

Front end

- HTML - structure. Semantic elements: header, nav, main, section, article, aside, footer, figure. Head holds metadata - meta charset, meta viewport, title, link, script. Forms: form with action and method, input, select, textarea, label, and enctype multipart/form-data for uploads.
- The anchor element with href makes a hyperlink; the link element in the head relates external resources and is not clickable.
- CSS - presentation. Selectors, the box model with content, padding, border and margin, specificity, the cascade, Flexbox, Grid, media queries, and custom properties.
- JavaScript - behaviour. DOM manipulation, events, the event loop with the call stack, task queue and microtasks, promises and async/await, fetch, closures, prototypes, ES modules.
- Accessibility: semantic markup, alt text, labels, keyboard operability, ARIA where semantics are insufficient, and WCAG contrast minimums.

Server side and PHP

- PHP is interpreted on the server, and only its output reaches the browser, so source and credentials are never exposed.
- Variables begin with \$, need no declaration, are dynamically typed, and their names are case sensitive while keywords and function names are not.

- Superglobals: \$_GET, \$_POST, \$_REQUEST, \$_SESSION, \$_COOKIE, \$_FILES, \$_SERVER, \$_ENV, \$GLOBALS.
- Double-quoted strings interpolate variables; single-quoted strings do not. Concatenation uses the dot operator.
- Database access with PDO or MySQLi, always using prepared statements with bound parameters - concatenating user input into SQL is what SQL injection exploits.
- Security essentials: validate and sanitise all input, escape all output to prevent cross-site scripting, use CSRF tokens on state-changing forms, hash passwords with password_hash which uses bcrypt, never with MD5 or SHA-1 alone, and set secure, HttpOnly, SameSite cookies.

Website development phases

- Planning and requirements; information architecture and content; design, from wireframe to mockup to prototype; development, front end and back end; testing - functional, cross-browser, responsive, performance, accessibility, security, SEO; deployment after testing; then maintenance and continuous improvement.
- Deployment happens after testing - a frequent examination point.
- SEO basics: title and meta description, heading hierarchy, alt text, clean URLs, sitemap.xml, robots.txt, canonical tags, structured data, mobile friendliness, page speed, HTTPS.

Most often tested

- head contains metadata such as charset and viewport.
- The anchor element creates a hyperlink.
- The server executes PHP and sends the resulting HTML.
- \$variable_name is valid; a space in the name is not; var is JavaScript.
- Deployment follows the testing phase.

12. Artificial Intelligence and Machine Learning

Foundations

- AI is the construction of systems that perform tasks normally requiring human intelligence. Sub-fields: search and problem solving, knowledge representation and reasoning, planning, learning, natural language processing, perception, robotics.
- Agent types: simple reflex, model-based reflex, goal-based, utility-based, learning.
- Turing test; weak or narrow AI versus strong or general AI; symbolic AI versus connectionist AI.

Search

- Uninformed: BFS, complete and optimal for unit costs; DFS, space efficient but not optimal; uniform-cost; depth-limited; iterative deepening, which combines DFS space with BFS optimality.
- Informed: greedy best-first, using $h(n)$; A*, using $f(n) = g(n) + h(n)$, which is optimal when h is admissible, meaning it never overestimates, and consistent.
- Local search: hill climbing, with local maxima, plateaux and ridges as failure modes; simulated annealing; genetic algorithms; beam search.
- Adversarial: minimax, and alpha-beta pruning, which reduces the effective branching factor without changing the result.
- Constraint satisfaction: backtracking with forward checking, arc consistency AC-3, and the minimum-remaining-values and least-constraining-value heuristics.

Knowledge representation and reasoning

- Schemes: production rules, semantic networks, frames, scripts, propositional and first-order logic, description logics and ontologies such as OWL and RDF.
- Inference: forward chaining, data driven, suits monitoring; backward chaining, goal driven, suits diagnosis. Resolution, unification, modus ponens.
- Uncertainty: probability and Bayes' theorem, Bayesian networks, certainty factors as in MYCIN, Dempster-Shafer theory, fuzzy logic.
- Non-monotonic reasoning, default logic, truth maintenance - needed because real rules have exceptions and new facts can invalidate old conclusions.
- Expert system architecture: knowledge base, inference engine, working memory, explanation facility, knowledge-acquisition facility, user interface. The separation of knowledge base from inference engine is the point of the design.
- PROLOG - declarative logic programming with facts, rules, unification and backtracking; used for expert systems, knowledge representation and NLP.

Learning

- Paradigms: supervised, with labelled data, for classification and regression; unsupervised, for clustering, dimensionality reduction and association rules; semi-supervised; reinforcement, learning from reward, as in Q-learning.
- Knowledge-level learning extends what the system can deduce - new facts, rules and concepts, acquired inductively or by being told. Symbol-level learning improves the efficiency of using existing knowledge - EBL, chunking, caching, rule reordering - and leaves the deductive closure unchanged.

- Explanation-Based Learning generalises the explanation of a single solved example into a reusable rule. It needs a domain theory, and can therefore learn from one example, unlike inductive methods. Its risk is the utility problem, where accumulated macro-rules cost more to match than they save.
- Inductive learning algorithms: decision trees - ID3, C4.5, CART, using information gain or the Gini index; rule induction; naive Bayes; k-NN; SVM; neural networks with back propagation; ensembles - bagging, random forest, boosting, AdaBoost, gradient boosting.
- Clustering: k-means, hierarchical agglomerative, DBSCAN. Dimensionality reduction: PCA, t-SNE.

k-NN in detail

- Instance-based, memory-based, lazy, non-parametric. No training phase; all work at prediction time. Prediction by majority vote among the k nearest neighbours, or the mean for regression.
- Distances: Euclidean, Manhattan, Minkowski, Hamming for categorical, cosine for sparse high-dimensional data.
- Performance factors: the value of k, which trades bias against variance; feature scaling, which is mandatory; the curse of dimensionality; irrelevant features, which k-NN cannot ignore; the metric; class imbalance; noise; training-set size; and prediction cost, mitigated by k-d trees, ball trees or approximate nearest neighbour.
- Choose an odd k for two classes to avoid ties, and select k by cross-validation.

Model evaluation

Confusion matrix:	predicted +	predicted -
actual +	TP	FN
actual -	FP	TN

$$\begin{aligned} \text{Accuracy} &= (TP + TN) / \text{total} \\ \text{Precision} &= TP / (TP + FP) \\ \text{Recall} &= TP / (TP + FN) \\ F1 &= 2PR / (P + R) \end{aligned}$$

- Accuracy is misleading on imbalanced data - a classifier that always predicts the majority class can score highly while being useless.
- Overfitting: low training error, high test error - the model has learned noise. Cures: more data, regularisation, simpler models, early stopping, dropout, cross-validation, pruning.
- Underfitting: high error everywhere - the model is too simple.
- The bias-variance trade-off: total error decomposes into bias, variance and irreducible noise. Small k in k-NN gives low bias and high variance; large k the reverse.
- k-fold cross-validation, and stratified folds when classes are imbalanced.

Perception and application areas

- Representation levels: raw signal, then numerical or statistical features, then symbolic and structured descriptions. A phoneme sequence is symbolic; a spatial arrangement of shapes is a structured description; average intensity is a numerical feature.
- Speech recognition - historically HMMs with the Viterbi and forward-backward algorithms, now end-to-end deep networks.
- Computer vision - segmentation, feature extraction, object recognition, CNNs.

- NLP - tokenisation, parsing, named entity recognition, sentiment analysis, machine translation, transformers and large language models.
- Knowledge graphs and retrieval-augmented generation combine curated symbolic knowledge, which is precise and auditable, with learned models, which are broad but opaque.

Most often tested

- Rules are the classic knowledge-base representation.
- An ontology is a formal, explicit specification of a shared conceptualisation.
- Decision trees are the standard inductive classification algorithm.
- EBL's advantage is needing far less training data, because it uses a domain theory.
- HMMs are historically associated with speech recognition.
- PROLOG is for expert systems and knowledge representation.
- k-NN is instance-based or memory-based learning.
- The curse of dimensionality is the deep problem for distance-based methods.

13. Mobile Application Development

Platforms and frameworks

- Native: Android with Kotlin or Java in Android Studio, using Jetpack Compose or XML layouts; iOS with Swift or Objective-C in Xcode, using SwiftUI or UIKit. Best performance and immediate access to new APIs, at the cost of two codebases.
- Cross-platform: Flutter with Dart, rendering its own widgets; React Native with JavaScript or TypeScript, mapping to native widgets; .NET MAUI with C#; Kotlin Multiplatform, sharing logic but not UI; Ionic with Capacitor, a web view.
- Progressive web apps - installable, offline capable through service workers, no store distribution.

Android and iOS essentials

- Android components: Activity, Fragment, Service, Broadcast Receiver, Content Provider. The manifest declares components and permissions. Activity lifecycle: onCreate, onStart, onResume, onPause, onStop, onDestroy, onRestart.
- iOS: view controllers, the app delegate and scene delegate, storyboards or SwiftUI views, and the app lifecycle states - not running, inactive, active, background, suspended.
- Storage: SQLite with Room on Android and Core Data on iOS, Realm, key-value preferences, files, and the keychain or keystore for secrets.
- Permissions are requested at run time on both platforms, and should be requested in context with an explanation.

UI design considerations

- Small screens and device fragmentation - responsive layouts in density-independent units, safe-area insets, foldables and split screen.
- Touch input - minimum targets 48 dp on Android and 44 pt on iOS, thumb-zone placement, no hover state, discoverable gestures.
- Platform conventions - Material Design versus the Human Interface Guidelines; navigation, typography, controls and elevation all differ.
- Performance - 60 fps means a 16 ms frame budget; keep work off the UI thread, recycle list views, size and cache images, animate only transform and opacity.
- Battery and network - avoid needless wake-ups, polling and background work; design offline-first with local state as the source of truth.
- States - every asynchronous view needs loading, success, empty and error, with a retry path.
- Accessibility - labels for screen readers, scalable text, 4.5:1 contrast, never colour alone, dark mode support, reduce-motion support.
- Internationalisation - text expansion of thirty to forty per cent, right-to-left mirroring, locale-dependent formats.
- Feedback and reversibility - pressed states, haptics, progress, and undo in preference to confirmation dialogues.

Delivery

- Testing: unit with JUnit or XCTest, UI with Espresso or XCTest, cross-platform with Appium or Detox, real-device farms such as Firebase Test Lab and BrowserStack.
- CI/CD: Gradle and the Xcode build system, Fastlane, GitHub Actions, Bitrise, Codemagic; distribution through the Play Console and App Store Connect, with staged rollout and beta tracks.
- Monitoring: Crashlytics or Sentry, analytics, performance tracing, and remote configuration with feature flags.

Most often tested

- Offline-first exists to ensure functionality without an internet connection.
- Flutter uses Dart; React Native uses JavaScript; native Android uses Kotlin; native iOS uses Swift.
- Touch targets: 48 dp Android, 44 pt iOS.

Appendix: Cross-subject facts worth memorising

Complexity

Linear search	$O(n)$	Binary search	$O(\log n)$
Stack push/pop	$O(1)$	LL head insert	$O(1)$
BFS / DFS (list)	$O(V + E)$	BFS / DFS (matrix)	$O(V^2)$
Merge sort	$O(n \log n)$ always		
Quick sort	$O(n \log n)$ avg, $O(n^2)$ worst		
Hash table	$O(1)$ avg, $O(n)$ worst		
BST	$O(\log n)$ balanced, $O(n)$ degenerate		
Heap insert/extract	$O(\log n)$	Build heap	$O(n)$

Digital logic

Two's complement	= invert, then add 1
$A + (AB)'$	= 1
Gray from binary	$G(\text{msb}) = B(\text{msb})$, $G_i = B_{i+1} \text{ xor } B_i$
Ring counter	n states with n flip-flops
Ripple adder delay	= stages $\times$ per-stage delay
Encoder $2^n \rightarrow n$	Decoder $n \rightarrow 2^n$
JK	$Q(\text{next}) = J.Q' + K'.Q$
T	$Q(\text{next}) = T \text{ xor } Q$

Operating systems

Turnaround	= completion - arrival
Waiting	= turnaround - burst
Paging	$\rightarrow$ internal fragmentation ($\leq$ half page/process)
Segmenting	$\rightarrow$ external fragmentation
Deadlock	= mutual exclusion + hold and wait + no preemption + circular wait

Networking

Hosts per subnet	= $2^{(32 - \text{prefix})} - 2$
Ports: FTP 21, SSH 22, Telnet 23, SMTP 25, DNS 53,	
HTTP 80, POP3 110, IMAP 143, HTTPS 443	
TCP header 20 bytes min, UDP header 8 bytes	

Software engineering

COCOMO	Effort = $a \times \text{KLOC}^b$
PERT	Expected = $(O + 4M + P) / 6$
$V(G)$	= $E - N + 2P$
Aim:	high cohesion, low coupling

Machine learning

Precision	= $TP / (TP + FP)$
Recall	= $TP / (TP + FN)$
F1	= $2PR / (P + R)$
Small k	$\rightarrow$ low bias, high variance
Large k	$\rightarrow$ high bias, low variance